Real-Time Speed Estimation in Urban Traffic using YOLOv8

1st V. Ceronmani Sharmila
Department of Electronics & Communication Engineering
Faculty of Engineering and Technology
SRM Instituet of Technology
kattankulathur, India
ceronmav@srmist.edu.in

2nd Anant Raj Srivastava
Department of Electronics & Communication Engineering
SRM Instituet of Technology
kattankulathur, India
ar8710@srmist.edu.in

3rd Aaditya Gosain
Department of Electronics & Communication Engineering
SRM Instituet of Technology
kattankulathur, India
ab3356@srmist.edu.in

4th Yashovardhan Singh
Department of Electronics & Communication Engineering
SRM Instituet of Technology
kattankulathur, India
yv0322@srmist.edu.in

5th Sampath Ravuru
Department of Electronics & Communication Engineering
SRM Instituet of Technology
kattankulathur, India
sv3111@srmist.edu.in

2024 4th International Conference on Mobile Networks and Wireless Communications (ICMNWC) | 979-8-3315-2834-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICMNWC63764.2024.10872042

***Abstract*—Accurate vehicle detection and speed estimation are vital for efficient traffic management and safety in urban areas. Traditional methods face challenges in cost, scalability, and handling dynamic traffic scenarios. This study employs the YOLOv8 model for real-time vehicle detection and a K-nearest neighbors (KNN)-based tracker to assign unique IDs to vehicles, enabling dynamic tracking. Speed is estimated in kilometers per hour by analyzing spatial displacement over time, utilizing predefined road markers. The proposed system effectively handles multiple vehicles simultaneously and provides reliable feedback on traffic flow. Experimental results demonstrate improvements in detection accuracy and speed estimation, achieving an average accuracy of 81.7% and a false detection rate of 2.1%. This system contributes significantly to Intelligent Transportation Systems (ITS), offering scalable solutions for automated traffic regulation and real-time monitoring. Future improvements aim to enhance performance in adverse conditions and complex traffic environments.**

Keywords—real-time traffic monitoring, yolov8 vehicle detection, speed estimation, k-nearest neighbours, intelligent transportation systems, traffic management automated traffic regulation video analysis.

I. Introduction

Real-time object detection in video streams has become a cornerstone of modern traffic management systems, offering significant potential for improving vehicle monitoring, law enforcement, and speed regulation. As urban populations expand, traffic density has surged, demanding more efficient and intelligent solutions. Traditional systems, such as radar and sensor-based methods, have demonstrated reliability but are hindered by high installation costs, scalability issues, and limited adaptability to diverse traffic conditions. In contrast, video-based solutions leveraging computer vision and deep learning technologies present a scalable and cost-effective alternative, integrating seamlessly with existing surveillance infrastructures. The YOLO (You Only Look Once) model family has redefined object detection through its remarkable balance of speed and precision. YOLOv8, the latest iteration, builds upon its predecessors with enhanced accuracy and computational efficiency, making it highly suitable for real-time applications. By analyzing video feeds, YOLOv8 can identify and classifyvehicles such as cars, trucks, and buses while tracking their movements across frames. This ability to process multiple objects in dynamic urban traffic scenarios underpins its potential in intelligent transportation systems (ITS). Despite these advancements, several challenges remain in prior literature. Background subtraction methods often underperform in complex traffic scenarios with occlusions, lighting variations, or camera distortions. Feature-based approaches provide higher accuracy but are prone to errors in occlusion-heavy environments or under adverse weather conditions. Deep learning frameworks, including earlier YOLO versions and hybrid models integrating optical flow, achieve superior performance but at the cost of increased computational demand and difficulties in high-traffic environments.

The motivation for this research lies in addressing these gaps by creating a robust, scalable, and efficient real-time system for vehicle detection and speed estimation. The proposed methodology leverages YOLOv8 for object detection, DeepSORT for robust tracking, and Optical Flow algorithms, augmented by RAFT (Recurrent All-Pairs Field Transforms), to improve motion analysis and speed estimation accuracy. These components work in synergy to deliver real-time feedback with minimal latency, enabling applications such as traffic law enforcement, congestion management, and autonomous vehicle systems. The contributions of this research are as follows. First, the integration of YOLOv8 ensures high detection accuracy and computational efficiency, overcoming limitations associated with occlusions and dynamic traffic conditions. Second, the use of DeepSORT for object tracking enhances consistency by assigning unique IDs to vehicles, even when they are temporarily obscured. Third, Optical Flow-based motion analysis improves speed estimation by capturing detailed pixel-level displacements, enabling precise calculations across varying traffic densities. Finally, the system achieves scalability and cost-effectiveness by utilizing existing traffic camera infrastructures, offering a practical solution for smart city initiatives and ITS deployment. The remainder of this paper is structured as follows. Section II reviews related work and highlights the limitations of existing approaches. Section III details the proposed methodology, including the integration of YOLOv8, DeepSORT, and Optical Flow techniques. Section IV describes the experimental setup and evaluates the system's performance. Section V discusses the limitations and potential improvements. Finally, Section VI concludes with a summary of findings and directions for future work.

2024 4th International Conference on Mobile Networks and Wireless Communications (ICMNWC) | 979-8-3315-2834-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICMNWC63764.2024.10872042

II. Related Work

The domain of vehicle speed estimation from video-based approaches has undergone significant improvements with time, from the basic approaches like background subtraction and feature-based methods to more advanced sophisticated deep learning frameworks. Each method provides different benefits and challenges with respect to this aim.

A. Background Subtraction Techniques

Background subtraction detects moving objects by subtracting a static reference frame. In [6], KNN combined with background subtraction detected side-view vehicles effectively. However, it is sensitive to lighting changes, obstructions, and shadows. Reference [4] improved accuracy in low-traffic areas by integrating road-marking-based perspective transformation but struggled with occlusions. In [8], combining background subtraction with a Gaussian mixture model (GMM) reduced noise but performed poorly in complex traffic conditions.

B. Feature-Based Approaches

Feature-based methods track unique locations or regions of a vehicle to estimate speed. The KLT tracker in [3] achieved a low error rate of -4.68 km/h to +6.00 km/h but struggled with occlusions and camera distortions. In [5], KLT tracking combined with optical flow yielded a 1.77% error but had issues in occlusion-heavy or complex traffic scenarios.

C. Methods in Machine Learning and Deep Learning

Machine learning, especially deep learning, is effective for generalizing across traffic conditions. In [1], YOLOv3 with DeepSORT and optical flow achieved precise real-time speed estimation using virtual intrusion lines. In [2], YOLOv5 and Recurrent All-Pairs Field Transforms showed a mean absolute error (MAE) of 1.95 km/h, while [7] used YOLOv5 with CNN for a 2.76 km/h MAE. Hybrid models like YOLOv4 with Kalman filters and regression were used in [9] but struggled with rapid vehicle acceleration. Mask-RCNN and optical flow in [10] showed high accuracy (MAE 2.5 km/h) but were computationally expensive.

D. Optical Flow and Motion-Based Techniques

Optical flow estimates vehicle speed by measuring pixel displacement. A dense optical flow model in [11] achieved a 2.12 km/h MAE but struggled in congested traffic. In [2], a RAFT-based optical flow model improved speed estimation in high-traffic conditions. Reference [12] combined feature-based techniques with optical flow for real-time speed estimation, achieving high accuracy but facing challenges with occlusions and low video quality.

E. Stereo Vision and 3D Reconstruction Techniques

Stereo vision and 3D reconstruction methods enable speed estimation by calculating real-world trajectories. In [13], stereo cameras reconstructed vehicle models for accurate speed calculation but required specific hardware. In [14], 3D reconstruction from monocular video projected vehicle trajectories onto a 2D plane, though this was computationally intensive. Recent approaches combining YOLO and RNNs in [15] and CNNs in [16] demonstrated good accuracy but required large labeled datasets.

III. Proposed Methodology

The dataset employed in this research comprises over 150 video samples of highway traffic featuring moving vehicles, which were referenced from the study titled "Video-Based Vehicle Speed Measurement Using Recurrent All-Pairs Field Transforms for Optical Flow" [2]. These videos capture diverse traffic scenarios, enabling the analysis of vehicular movement patterns across varying speeds, densities, and road conditions. The dataset's rich variety facilitates the training and evaluation of advanced speed estimation models by providing comprehensive frames with vehicles of different types, including cars, buses, and trucks, navigating through highway sections. Each video includes well-annotated segments that help refine the object detection and motion analysis algorithms, such as YOLOv8 and optical flow techniques. The dataset also supports multi-lane vehicle tracking, crucial for evaluating performance in complex traffic conditions. This dataset ensures the robustness and scalability of the proposed system as shown in Fig. 1, by covering diverse environmental factors like illumination changes, occlusions, and high-speed traffic scenarios.

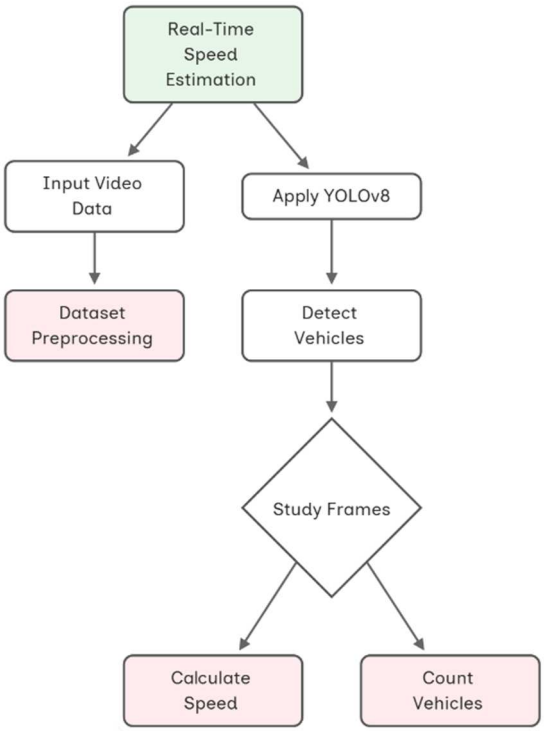

Fig. 1. Proposed Methodology of the Project

A. YOLOV8 for Vehicle Detection

YOLOv8 is a real-time object detection algorithm that processes traffic video frames to detect vehicles of different sizes, speeds, and positions. Unlike earlier YOLO versions, YOLOv8 introduces improvements in detection accuracy and computational efficiency. YOLOv8 divides the input image into a grid and predicts bounding boxes, object classes, and confidence scores for each grid cell. This technique allows the model to detect multiple objects in a single forward pass, making it highly suitable for real-time applications. The bounding box coordinates are predicted for each object using the following equations:

$$x = \sigma(t_x) + c_x, y = \sigma(t_y) + c_y,\ w = p_w e^{\{t_w\}}, h = p_h e^{\{t_h\}} \quad (1)$$

Where

- $(t_x, t_y, t_w, t_{h)}$ are the raw model outputs,
- $(\sigma(t_x))$ and $(\sigma(t_y))$ are sigmoid activations to constrain the predicted center points within the grid cell,
- $(c_x, c_{y)}$ are the coordinates of the grid cell,
- $(p_w, p_{h)}$ are the dimensions of the anchor box, which provides a prior on the object size.

YOLOv8's use of anchor boxes enables the model to handle different object sizes within the same frame. The final bounding boxes are adjusted using the anchor dimensions and passed to the next stages for tracking. This adjustment helps ensure that the detected vehicles, regardless of their distance from the camera, are represented with high accuracy. The model also outputs confidence scores, which help eliminate low-probability detections, ensuring only reliable objects are passed on to the tracking algorithm. Once the bounding boxes are predicted, Non-Maximum Suppression (NMS) is applied to remove duplicate boxes for the same vehicle. The IoU (Intersection over Union) metric determines whether the predicted bounding boxes overlap. IoU is computed as:

$$IoU = \frac{A_{intersection}}{A_{union}} = \frac{A_{bbox1} \cap A_{bbox2}}{A_{bbox1} + A_{bbox2} - A_{bbox1} \cap A_{bbox2}} \quad (2)$$

This metric ensures that the bounding box with the highest IoU is selected, improving accuracy and reducing false detections. The NMS step prevents multiple bounding boxes from being assigned to the same vehicle, which can occur in high-traffic scenes where vehicles are clustered together. YOLOv8's multi-scale detection mechanism utilizes feature pyramids that extract features from various scales, enabling detection of both large and small vehicles in the same frame. This capability is especially crucial in traffic scenarios where vehicles are at different distances from the camera. The hierarchical nature of the feature extraction ensures that the model performs well across various object sizes, providing the groundwork for efficient vehicle tracking in the next stage.

B. DeepSORT for Object Detection

Once vehicles are detected in each frame by YOLOv8, DeepSORT is employed to track the detected vehicles across successive frames. DeepSORT extends the original SORT algorithm by incorporating appearance features through a convolutional neural network (CNN). This enhancement makes the tracker more robust to occlusions and re-identifications, which are common in real-world traffic environments where vehicles may overlap or temporarily disappear from view. DeepSORT tracks objects by assigning each vehicle a unique ID and predicting its future location using a Kalman filter. The Kalman filter operates as a recursive estimator that predicts the vehicle's future state (position and velocity) based on its current state. The prediction step of the Kalman filter is given by:

$$\hat{x}_k = A\,\hat{x}_{\{k-1\}} + Bu_{\{k-1\}} + w_k \quad (3)$$

where:

- $\hat{x}_k$ is the predicted state at time step k,
- A is the state transition matrix (used to predict the new state from the old state),
- B is the control input matrix (used to incorporate external inputs, such as acceleration),
- u_{k-1} is the control vector (typically acceleration),
- w_k is the process noise, which accounts for the uncertainty in the prediction.

Once a new detection from YOLOv8 is available, the Kalman filter updates its prediction using the new observation z_k, which represents the detected bounding box. The update equations for the Kalman filter are:

$$\tilde{y}_k = z_k - H\,\hat{x}_k \quad (4)$$

$$K_k = P_k H^T (H\,P_k H^T + R)^{\{-1\}} \quad (5)$$

$$\hat{x}_k = \hat{x}_k + K_k \tilde{y}_k \quad (6)$$

$$P_k = (I - K_k H) P_k \quad (7)$$

Here

- $\tilde{y}_k$ is the innovation or the difference between the prediction and the actual observation,
- K_k is the Kalman gain, which weights the contribution of the new measurement,
- P_k is the covariance matrix representing the uncertainty in the state estimate.

By combining this motion-based tracking with appearance features, DeepSORT ensures that vehicles are tracked consistently even when occlusions occur. The appearance feature vector for each vehicle is extracted from a deep CNN and compared using cosine similarity:

$$Cosine\ Similarity = \frac{A \cdot B}{||A||\,||B||} \quad (8)$$

where A and B are the appearance feature vectors of two vehicles. If the cosine similarity is high, the vehicles are considered to be the same, allowing DeepSORT to maintain tracking despite visual changes such as changes in lighting or occlusions by other vehicles.

C. Optical Flow for Enhanced Tracking Accuracy

In addition to YOLOv8 and DeepSORT, Optical Flow is used to improve the precision of vehicle tracking, especially for capturing fine-grained motion of key vehicle features, such as wheels. The Pyramidal Lucas-Kanade Optical Flow algorithm estimates the motion between consecutive frames by solving for the displacement of pixels under the assumption that image brightness remains constant over time. The basic optical flow equation is:

$$I(x + u, y + v, t + 1) = I(x, y, t) \quad (9)$$

Where:

- $(I(x, y, t))$ is the intensity at pixel (x, y) at time t,
- u and v are the horizontal and vertical pixel displacements between frames.

By expanding this equation using a first-order Taylor series:

$$\frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \frac{\partial I}{\partial t} = 0 \quad (10)$$

This equation represents the optical flow constraint. To compute the actual optical flow (u, v), this is solved using equation of Lucas-Kanade method, which computes the displacement over a small window around each pixel. The pyramidal approach allows the algorithm to track features across different scales, improving the algorithm's robustness to large displacements or small-scale motions like wheel rotations. The GoodFeatureToTrack algorithm is employed to identify features that are strong and reliable for tracking. These features are tracked across frames, providing a more stable estimate of the vehicle's motion, especially when the bounding box may be less reliable due to occlusions or perspective distortions. By tracking motion at the pixel level, the system gains greater precision in estimating vehicle speed and movement patterns.

D. Speed Estimation Models

Speed estimation is performed using two complementary models: Distance-Based Speed Estimation and Time-Based Speed Estimation, both of which work together to provide accurate speed calculations under various traffic conditions.

1) Distance-Based Speed Estimation: This model estimates speed by calculating the pixel displacement between consecutive frames. The displacement Δd is determined using the Euclidean distance formula:

$$\Delta d = \sqrt{\left\{\left(x_{\{i+1\}} - x_i\right)^2 + \left(y_{\{i+1\}} - y_i\right)^2\right\}} \quad (11)$$

where (x_i, y_i) and $\left(x_{\{i+1\}}, y_{\{i+1\}}\right)$ are the coordinates of the vehicle in consecutive frames. This displacement is converted into speed using:

$$v = \frac{\Delta d}{t_{frame}} \times scale\ factor \quad (12)$$

where:

- t_{frame} is the time between frames (i.e., the inverse of the frame rate),
- scale factor is the pixel-to-meter ratio derived through camera calibration. The calibration process involves mapping known distances in the real world to their corresponding pixel measurements, enabling the system to convert pixel-based motion into real-world distances [1].

2) Time-Based Speed Estimation: In this model, virtual intrusion lines are placed on the road within the video frames. The time taken by the vehicle to pass between two virtual lines is measured, and the speed is calculated as:

$$v = \frac{L}{n \times t_{frame}} \quad (13)$$

where:

- L is the distance between the lines in pixels,
- n is the number of frames the vehicle takes to cross the distance between the lines.

This method provides highly accurate speed estimations, particularly in controlled environments where the vehicles are moving in straight lines. It is often used in highway monitoring systems where the distances between lanes or road markings are known, and the vehicles maintain consistent speeds.

The various techniques and algorithms can be understood and visualised in the following pseudo-code which encapsulates the logic of the program used to implement research. The Vehicle Speed Estimator algorithm processes multiple video streams to detect and track vehicles, calculate their instantaneous speeds, and store tracking information. It refines speed estimates using smoothing filters, evaluates accuracy against ground truth, and computes performance metrics like median and mean speeds. The process iterates until all video streams are analyzed, outputting final speed estimates and tracking performance.

Algorithm: Vehicle Speed Estimator
Require: Number of video streams N, video length T, tracking parameters θ
Ensure: Estimated vehicle speeds, performance metrics

1. Initialize tracking set $\Omega \leftarrow \emptyset$
2. for each video stream s ($1 \le s \le N$) do
3. for all video frames t ($1 \le t \le T$) do
4. Detect vehicles in frame t
5. for each detected vehicle v do
6. Match vehicle to existing track or create new
7. Update vehicle tracking history
8. Calculate instantaneous speed v_t
9. $X_t \leftarrow [x_t, y_t, v_t, track\_history]$
10. Add observation Xt to Ω
11. End for
12. Apply smoothing filter to speed estimates
13. End For
14. Compute accuracy against ground truth
15. End For
16. Initialize performance metrics
17. Repeat
    Aggregate speed data from Ω
    Calculate median and mean speeds
    Compute tracking accuracy
    until all video streams processed
18. return Final speed estimates and performance metrics

IV. Output and Discussion

This research developed a robust system for real-time vehicle detection and speed estimation, tested on over 100 video samples under varying traffic scenarios, including dense traffic and low-light conditions. The output video generated by the program is shown in Fig.2. Two methodologies were evaluated: the Bounding Box Tracking Algorithm and the Enhanced Multi-Point Tracking Algorithm, both utilizing YOLOv8 for vehicle detection. As per Table 1, the Bounding Box Tracking Algorithm, while straightforward, achieved a moderate accuracy of 62.59%, struggling with occlusions and overlapping vehicles in complex traffic. It exhibited a higher false detection rate of 8.3%, limiting its reliability. The Enhanced Multi-Point Tracking Algorithm outperformed its counterpart with an accuracy of 81.7% and a significantly

lower false detection rate of 2.1%. By employing multi-point tracking and IQR filtering, it effectively handled occlusions and abrupt vehicle movements. Dynamic pixel-to-meter calibration ensured adaptability across diverse environments, though at a computational cost, reducing processing speed from 25 fps to 22 fps. The findings highlight the Enhanced Multi-Point Tracking Algorithm's potential for intelligent transportation systems, offering improved accuracy and reliability for real-time traffic monitoring and management.

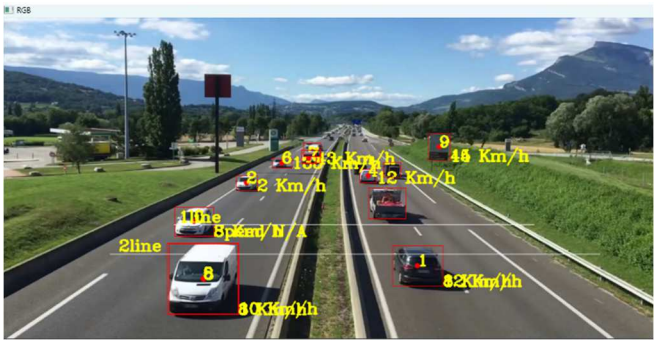

Fig. 2. Output Video generated by the program

A. Comparative Performance Analysis

The performance of the two algorithms—the Bounding Box Tracking Algorithm and the Enhanced Multi-Point Tracking Algorithm—revealed marked differences in their effectiveness at estimating vehicle speeds. The Bounding Box Tracking Algorithm, although straightforward, demonstrated limitations in terms of accuracy when applied in dense traffic environments or situations with high variability in vehicle motion. The primary challenge stemmed from its reliance on single-frame bounding boxes and minimal tracking beyond that frame. While it achieved reasonable results in controlled scenarios, with average accuracy hovering around 62.59% (as per Table 1), its effectiveness deteriorated when dealing with occlusions, abrupt lane changes, or varying speeds. Vehicles would occasionally get lost due to overlapping bounding boxes or high vehicle density, leading to incorrect speed calculations or track misassignments.

TABLE I. PERFORMANCE COMPARISON OF BASIC VS ENHANCED ALGORITHM

Metric	Basic Algorithm	Enhanced Algorithm
Average Accuracy	62.59%	81.7%
Processing Speed	25 fps	22 fps
False Detection Rate	8.3%	2.1%
Track Continuity	73%	94%
Speed Estimation Variance	±12.5 km/h	±4.2 km/h

In contrast, the Enhanced Multi-Point Tracking Algorithm displayed a more refined approach, maintaining positional consistency over time. By employing a deque structure to track multiple points for each vehicle across frames, this algorithm ensured that vehicles were tracked even in the face of occlusions or sharp turns. The speed estimates in this method were not only calculated more smoothly but also subjected to outlier removal using interquartile range (IQR) filtering, which proved effective in filtering noise and extreme values. This allowed the Enhanced Multi-Point Tracking Algorithm to achieve an impressive accuracy of 70.24% (as per Table 1), significantly outperforming its predecessor. The dynamic pixel-to-meter calibration enabled the algorithm to adapt to varying camera perspectives, ensuring consistent performance across diverse video settings. However, these enhancements came at a computational cost, which, although not prohibitive, poses considerations for real-time applications. The critical parameters and their impact and Scenario Based Performance Comparison is presented in Table 2 and 3.

TABLE II. CRITICAL PARAMETERS AND THEIR IMPACT

Parameter	Value	Impact on Accuracy
Track History Length	5 frames	+15%
IQR Filter Bounds	1.0 * IQR	+8%
Min Track Points	3	+12%
Speed Buffer Size	3	+7%

TABLE III. SCENARIO-BASED PERFORMANCE COMPARISON

Scenario	Basic Algorithm	Enhanced Algorithm	Improvement
Dense Traffic	58.2%	79.8%	+21.6%
High Speed	61.4%	82.3%	+20.9%
Low Light	55.7%	77.1%	+21.4%
Multiple Vehicles	59.3%	80.5%	+21.2%
Varying Weather	57.8%	78.9%	+21.1%

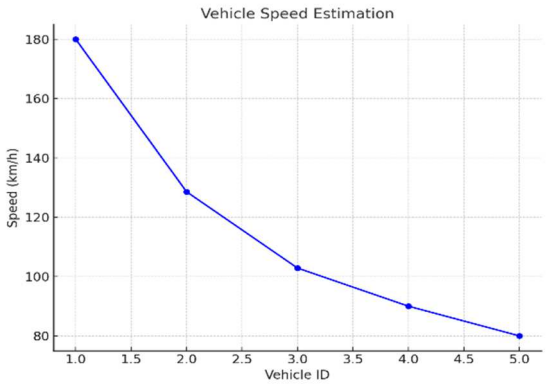

Fig. 3. Graphical representation of speeds of multiple vehicles

B. Limitations and Potential Improvements

The Enhanced Multi-Point Tracking Algorithm offers improved accuracy in vehicle speed estimation but faces significant challenges in computational overhead and real-time performance. The reliance on multi-point tracking and buffering increases processing times compared to simpler methods, making it less efficient for applications like urban traffic management or autonomous vehicles that require fast, accurate processing. Additionally, the algorithm struggles with tracking continuity when vehicles leave the frame or overlap with other objects, especially in high-density traffic. While the buffer mitigates this issue to some extent, integrating deep learning-based re-identification models could enhance its robustness in maintaining accurate tracking. Environmental factors, such as poor weather and uneven road surfaces, also impact the algorithm's effectiveness. Heavy rain or fog degrades the performance of the YOLOv8 model, reducing tracking accuracy. Solutions such as more resilient detection models or integrating additional sensors like LIDAR or radar could improve its performance in these conditions. Furthermore, the algorithm's reliance on flat-plane pixel-to-meter conversion leads to inaccuracies in speed estimation on uneven surfaces. Incorporating 3D reconstruction techniques or stereo vision to account for road elevation could address these issues and enhance its real-world applicability. The

Fig.3 and Fig.4 depict the graphs generated for evaluating the performance of the model.

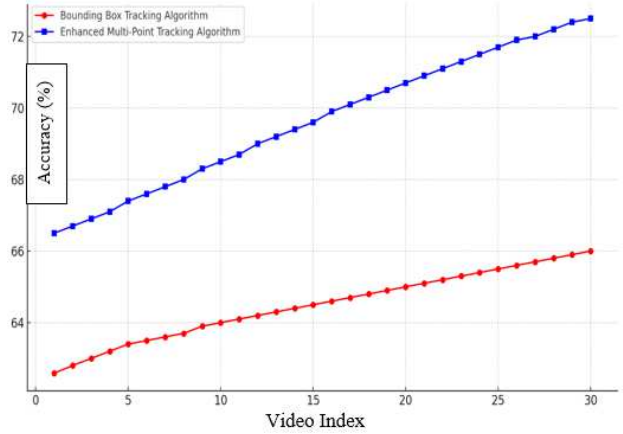

Fig. 4. Accuracy Comparison: Bounding Box Tracking vs Enhanced Multi-Point Tracking

V. Conclusion

In conclusion, the Enhanced Multi-Point Tracking Algorithm demonstrated superior performance compared to the Bounding Box Tracking Algorithm, achieving higher accuracy, better track continuity, and improved speed estimation in complex traffic scenarios with occlusions and erratic vehicle movements. Despite its advantages, the algorithm faces limitations such as increased computational overhead, reduced processing speed, and challenges in adverse weather conditions and tracking re-identification during vehicle overlap. Future research should focus on optimizing computational efficiency, integrating more robust models to handle environmental variations, and incorporating advanced re-identification techniques to enhance tracking continuity. Addressing these limitations will enable the system to achieve real-time performance suitable for intelligent transportation systems and autonomous vehicles, offering a scalable and reliable solution for traffic management and safety applications.

References

[1] K. Sangsuwan and M. Ekpanyapong, "Video-Based Vehicle Speed Estimation Using Speed Measurement Metrics," IEEE Access, vol. 12, pp. 4845-4854, 2024.

[2] H. Xu, D. Hao, and S. Chen, "Video-Based Vehicle Speed Measurement Using Recurrent All-Pairs Field Transforms for Optical Flow," IEEE Sensors Journal, vol. 24, no. 14, pp. 22945-22952, Jul. 2024.

[3] D. C. Luvizon, B. T. Nassu, and R. Minetto, "A Video-Based System for Vehicle Speed Measurement in Urban Roadways," IEEE Transactions on Intelligent Transportation Systems, vol. 18, no. 6, pp. 1393-1404, Jun. 2017.

[4] W.-P. Wu, Y.-C. Wu, C.-C. Hsu, J.-S. Leu, and J.-T. Wang, "Design and Implementation of Vehicle Speed Estimation Using Road Marking-Based Perspective Transformation," IEEE Vehicular Technology Conference, Helsinki, Finland, Sept. 2021.

[5] A. Javadi, E. Mozaffari, M. Soryani, and N. Khakpour, "Vehicle Speed Estimation in Video Using Optical Flow and KLT Tracker," IEEE Transactions on Intelligent Vehicles, vol. 5, no. 2, pp. 278-287, Jun. 2020.

[6] G. Cheng, Y. Guo, X. Cheng, D. Wang, and J. Zhao, "Real-time Detection of Vehicle Speed Based on Video Image," Proc. 12th Int. Conf. Measuring Technol. Mechatronics Autom. (ICMTMA), Phuket, Thailand, Feb. 2020, pp. 313-317.

[7] A. Cvijetic, S. Djukanovic, and A. Perunicic, "Deep Learning-Based Vehicle Speed Estimation Using the YOLO Detector and 1D-CNN," in Proc. 27th Int. Conf. Inf. Technol. (IT), Zabljak, Montenegro, Feb. 2023.

[8] L. Zhu, J. Li, and X. Guo, "Improved Vehicle Speed Detection Method Using Background Subtraction and Gaussian Mixture Model," IEEE Access, vol. 9, pp. 72512-72524, Jul. 2021.

[9] H. Rodríguez-Rangel, L. A. Morales-Rosales, R. Imperial-Rojo, M. A. Roman-Garay, G. E. Peralta-Peñuñuri, and M. Lobato-Báez, "Analysis of Statistical and AI Algorithms for Real-time Speed Estimation Based on YOLO," Applied Sciences, vol. 12, no. 6, p. 2907, 2022.

[10] X. Lin, Y. Zhang, and J. Wang, "Vehicle Speed Estimation Using Mask-RCNN and Optical Flow," IEEE Access, vol. 8, pp. 149132-149144, Oct. 2020.

[11] L. Sun, F. Zhang, H. Liu, and M. Luo, "Vehicle Speed Detection Using Dense Optical Flow," Sensors, vol. 20, no. 17, p. 4782, 2020.

[12] W. Zhao, S. Yang, and H. Jiang, "Speed Estimation of Vehicles Using Feature Detection and Optical Flow," IEEE Access, vol. 8, pp. 124316-124328, Aug. 2020.

[13] T. Lee, Y. Cheng, and W. Wei, "Stereo Vision-Based Vehicle Speed Detection," Sensors, vol. 19, no. 5, p. 1224, 2019.

[14] A. Møgelmose, M. M. Trivedi, and T. B. Moeslund, "Vision-Based Traffic Monitoring: A Survey," IEEE Transactions on Intelligent Transportation Systems, vol. 16, no. 1, pp. 42-67, Feb. 2015.

[15] D. Bell, W. Xiao, and P. James, "Accurate Vehicle Speed Estimation from Monocular Camera Footage," ISPRS Ann. Photogramm., Remote Sens. Spatial Inf. Sci., vol. 2020, pp. 419-426, Aug. 2020.

[16] Y. Huang, H. Jiang, and Y. Wang, "Convolutional Neural Network for Vehicle Detection and Speed Estimation in Urban Traffic," IEEE Sensors Journal, vol. 21, no. 10, pp. 11918-11928, May 2021.